Shopify Ghostferry: Live MySQL Shard Migration

Online Tenant Migrations, Row-Based Binlog Streaming & Sub-500ms Cutovers

DATA PLATFORM
MYSQL SHARDING

1. The Multi-Tenant Sharding Architecture at Shopify

Shopify powers millions of merchants processing billions in Gross Merchandise Volume (GMV). From its earliest architectural iterations, Shopify structured its persistence tier around a **shared-nothing, tenant-partitioned MySQL topology**. All primary tables (orders, products, customers, checkouts) carry a composite primary key or foreign key prefixed by `shop_id`.

The Noisy Neighbor Flash-Sale Problem

In shared multi-tenant pods, dozens of small merchants share a single physical MySQL primary. When an enterprise merchant (e.g., Gymshark or a celebrity influencer) launches an unannounced flash sale, orders surge 100x within seconds. The shard exhausts CPU and connection pools, degrading checkout latency for innocent neighboring stores. Shopify must migrate the hot tenant to a dedicated pod without taking their storefront offline.

2. Ghostferry Core Architecture & Dual Pipeline

To migrate active production databases without scheduled maintenance windows, Shopify engineered **Ghostferry**—a high-throughput Go engine that reconciles concurrent bulk copying with real-time replication:



3. The Interleaving Overwrite Race Condition

In a live database, moving historical records while new orders arrive creates an out-of-order race hazard:

- If Ghostferry reads a batch of rows from the Source, but a customer updates row 200 before that batch arrives at the Target, replaying the binlog could result in the older batch snapshot overwriting the newer binlog update.
- Ghostferry resolves this by establishing binlog streaming *before* initiating bulk table copying, tracking target row versions, and applying strict write-invalidation barriers.

4. End-to-End Cutover Protocol: Five Seamless Steps

Once Ghostferry completes historical data copying and the real-time binlog replication lag drops below 100ms, the system initiates an atomic cutover with minimal client friction:

```
Step 1: Application Tenant Lock (~50ms)
        Shopify's routing proxy sets the specific migrating shop to "READ-ONLY". Inbound
        writes (e.g. checkout POST requests) are temporarily delayed at the HTTP gateway.

Step 2: Binlog Tail Catch-Up (~150ms)
        Ghostferry waits for the Target to consume the remaining few binlog events until
        the Target binary log coordinates match the Source exactly.

Step 3: Verification & Checksums (~50ms)
        Ghostferry verifies row counts and cryptographic checksums across critical tables.

Step 4: Atomic Routing Switch (~10ms)
        The routing proxy updates its in-memory routing table (or ZooKeeper / Vitess map),
        pointing the shop's database connection pool to the Target Pod.

Step 5: Release Tenant Lock (~10ms)
        The shop transitions back to READ-WRITE on the Target Shard. Delayed HTTP requests
        execute immediately. Total customer-perceived pause: < 500 milliseconds!
```

5. Adaptive Throttling & Resource Protection

Ghostferry runs directly against production primary and replica databases. To ensure it never degrades active buyer checkout transactions, Ghostferry monitors database health metrics in real time:

Replica Lag Monitoring
Monitors `Seconds_Behind_Master` on downstream read replicas. If replica lag exceeds 2 seconds, Ghostferry pauses table reading threads to allow replicas to catch up.

Thread Concurrency Throttling
Monitors `Threads_running` in MySQL. If active connections spike past 80% capacity due to unexpected flash traffic, Ghostferry backs off exponentially.

6. Live Migration Approaches Comparison

Migration Strategy	Downtime Duration	Write Safety	Operational Complexity
Shopify Ghostferry	**< 500 ms (Write stall only)**	100% (RBR binlog tailing + invalidation)	Automated, tenant-scoped
Scheduled Offline Window	2–8 hours	100% (DB stopped)	Severe revenue loss; customer outrage
Dual-Writing in Application	Zero downtime	High risk of silent divergence & dual-writes	Massive code changes across all models
MySQL Native Replication	5–30 seconds	Can migrate entire instance, not individual shops	Cannot split shared multi-tenant tables

7. Cryptographic Data Verification & Rollback Safety

In financial and e-commerce systems, data corruption is catastrophic. Ghostferry incorporates multilayered verification and immediate abort capabilities:

- **Deterministic Chunk Checksumming:** Iterates over row chunks computing CRC32/MD5 hashes over all non-generated columns on both Source and Target, ensuring zero bit flips or encoding discrepancies.
- **Zero-Cost Abort Prior to Cutover:** Because the Source database remains the active system of record throughout the entire migration process, Ghostferry can be safely terminated at any instant before Step 4 without customer impact.
- **Reverse Replication Fallback:** Immediately after cutover, Ghostferry configures reverse replication from Target back to Source. If an unexpected bug arises on the Target shard within the first hour, traffic can be instantly routed back to Source with zero data loss.

8. Key Architectural Takeaways for Live Data Migrations

Engineering Principles Distilled from Shopify Ghostferry

- **Binlog Tailing Decouples Data Size from Downtime:** The duration of data copying (whether 10 GB or 10 TB) has zero effect on downtime. Only the final binlog draining lag dictates the cutover pause.
- **Queueing Beats Dropping During Cutover:** Pausing write queries for 300ms at the HTTP load balancer level is completely unnoticeable to a human clicking a checkout button, avoiding hard 500 errors.
- **Design for Tenant-Level Granularity:** Structuring databases with explicit tenant keys (`shop_id`) allows fine-grained, surgical shard migrations without touching unrelated platform traffic.
- **Always Maintain a Reverse Replication Safety Net:** Never sever ties with the source database immediately after cutover. Reverse replication guarantees an escape hatch if latent bugs surface under production load.